

LARS JANSEN

Senior Software Engineer

Assen, Drenthe, Netherlands | +31 97058049397 | larsjansen0521@outlook.com | linkedin.com/in/lars-jansen-0a37b8393

PROFESSIONAL SUMMARY

Senior Software Engineer with 10+ years building data-heavy SaaS and workflow platforms, delivering **Python** backend services (Django 2.2–4.2, DRF) and **React** frontends (16–18) end-to-end. In the transportation & fleet-tracking industry I shipped features on Django 3.2–4.2 while hardening APIs with **JWT/OAuth2** and rate limits. I design **REST APIs**, async job systems (**Celery** 4–5), and durable data layers (**PostgreSQL** 9.6–15, **Redis** 5–7), then ship via CI/CD, Docker, and cloud automation. Known for rescuing legacy code: debug incidents, add **OpenTelemetry** traces, optimize slow queries, and migrate modules to containers without breaking SLAs.

SKILLS

- Languages: Python 3.9–3.12, **JavaScript (ES2019–ES2024)**, **TypeScript 4.8–5.5**, **SQL**
- Backend & APIs: FastAPI 0.95–0.110, **asyncio**, **REST (OpenAPI 3.0/3.1)**, **WebSockets/SSE**, **background workers**, **schema validation**, **auth** (OAuth2/OIDC, JWT, RBAC)
- Frontend: React 17–18 (**hooks**, **performance profiling**), Angular 12–18 (**RxJS**, **state patterns**), **dashboards**, **responsive UI architecture**
- Data: PostgreSQL 12–16 (**indexes**, **migrations**, **EXPLAIN**), **Redis 6–7 (caching)**, **event-driven query patterns**
- Cloud & DevOps: **AWS (API Gateway, Lambda, ECS/EKS, S3, RDS, CloudWatch, IAM)**, Docker, Kubernetes, **Helm**, **CI/CD with GitHub Actions**
- Testing & Quality: **Pytest**, **Jest**, **React Testing Library**, **Cypress/Playwright**, **contract testing**, **load testing**, **linting/formatting gates**
- Tooling: **Git**, **trunk-based workflows**, **code reviews**, **feature flags**, **incident runbooks**

WORK HISTORY

Senior Software Engineer RavenTrack

August 2021 to November 2025

- Owned a multi-tenant **Django** (3.2–4.2) platform for fleet events, focusing on reliability, schema consistency, and predictable release cadence under tight SLAs.
- Built and versioned **REST APIs** with DRF, adding strict validation and backward-compatible serializers to prevent breaking changes for mobile and partner clients.
- Designed async processing with **Celery** (4–5) and Redis, eliminating duplicate job execution by enforcing idempotency keys and dead-letter retry policies.
- Resolved a recurring “phantom duplicates” bug by tracing race conditions in task acknowledgements and introducing transactional outbox writes in **PostgreSQL**.
- Migrated legacy Django 2.x modules to Django 4.x patterns, replacing deprecated middleware, updating authentication backends, and modernizing settings safely.
- Implemented a new real-time tracking view using **React** (17–18) with server pagination, reducing page freezes by removing heavy client-side filtering.
- Fixed a high-severity memory leak in a React table by stabilizing memoized selectors and removing accidental event-listener reattachment loops.
- Introduced **OpenTelemetry** tracing across API, Celery workers, and DB calls, cutting incident triage time by correlating slow endpoints to query plans.
- Optimized slow queries by rewriting ORM hotspots, adding partial indexes, and using EXPLAIN to validate reductions in sequential scans on large tables.
- Led a gradual migration from Webpack 4 to 5, then to Vite where appropriate, keeping bundle output stable and preventing regressions in legacy pages.
- Implemented feature flags for high-risk releases, enabling safe rollout of a new “geofence alerts” feature with instant kill-switch capability.
- Improved security posture by enforcing **JWT** rotation, tightening refresh rules, and adding audit logs for permission changes and admin actions.
- Containerized critical services with Docker and standardized local dev via compose, reducing “works on my machine” bugs across the team.
- Partnered with QA to add contract-style API tests and deterministic fixtures, catching breaking payload changes before they reached staging.
- Mentored engineers on debugging production issues by combining logs, traces, and DB metrics, creating a repeatable runbook for common failures.

Software Engineer Infinite Loop

June 2018 to July 2021

- Developed backend features in **Python** 3.x and **Django** (2.2–3.2), building operational workflows that depended on consistent state transitions.

- Created DRF endpoints for order and dispatch flows, enforcing permission checks and rate limits to protect core services from noisy clients.
- Built a background ingestion pipeline with **Celery** and Redis that processed partner feeds, validating data and quarantining malformed records.
- Solved a hard-to-reproduce “stuck in pending” bug by adding database-level constraints and compensating actions when external callbacks failed.
- Implemented caching for read-heavy endpoints using **Redis**, reducing database load while keeping cache invalidation explicit and measurable.
- Delivered UI features with **React** 16–17, refactoring class components into hooks to reduce duplicated state and improve testability.
- Tracked down a production 500 spike to a timezone conversion edge case, adding regression tests and normalizing timestamps at API boundaries.
- Introduced structured logging and Sentry error grouping, turning noisy exceptions into actionable categories tied to specific releases.
- Contributed to a Docker-based deployment workflow and CI checks, preventing broken migrations from being merged into the main branch.
- Improved search and filtering performance by shaping Elasticsearch queries and limiting expensive wildcard patterns that caused cluster pressure.
- Collaborated with product to ship a new “bulk import” feature, including resumable uploads and clear user-facing error reporting for bad rows.

Software Developer Datamart

October 2015 to May 2018

- Maintained a legacy **Django** (1.8–2.0) monolith and gradually modernized it without disrupting daily operations for internal users.
- Implemented new reporting screens and forms, balancing server-rendered templates with early **React** 15–16 widgets where interactivity mattered.
- Fixed recurring migration failures by squashing conflicting schema changes and standardizing naming conventions for safer incremental rollouts.
- Improved database performance in **MySQL** 5.7 and PostgreSQL 9.6 by adding indexes, removing N+1 queries, and batching expensive exports.
- Built reliable scheduled jobs for data sync, adding retries and alerting so “silent failures” became visible and quickly recoverable.
- Resolved intermittent file-upload errors by tuning Nginx limits and rewriting upload handling to stream content instead of buffering in memory.
- Reduced UI regressions by adding pragmatic unit tests and smoke checks, catching template rendering failures before deployments.
- Supported a gradual authentication upgrade path, cleaning up session handling and tightening access control for admin-only tooling.
- Partnered with ops to stabilize deployments through repeatable scripts and environment parity, reducing rollback frequency after releases.

EDUCATION

Bachelor’s degree in Computer Science University of Amsterdam

2011 to 2015

- Built strong foundations in software engineering, data structures, and systems design, with coursework that supports designing reliable services and data-driven applications.

PROJECTS

Fleet Insight Dashboard (Django + React)

- Built a near real-time dashboard that aggregates vehicle events and surfaces anomalies, using DRF for APIs and React 18 for interactive drill-down views.
- Improved operational response by adding trace-linked error panels and query-optimized timelines, so support could diagnose issues in minutes instead of hours.